```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import time
```

Question 1 (20 points) — Pipeline and EDA

(a) (4 pts) Load the provided cleaned train/test CSVs and keep the time split.

```
train_df = pd.read_csv('sample_data/hw3_train.csv')
train_df
```

	acc_now_delinq	addr_state	annual_inc	application_type	collections_12_mths_ex_med	delinq_2yrs	dti	earlie
0	0.0	AZ	24000.0	INDIVIDUAL	0.0	0.0	27.65	
1	0.0	GA	30000.0	INDIVIDUAL	0.0	0.0	1.00	
2	0.0	IL	12252.0	INDIVIDUAL	0.0	0.0	8.72	
3	0.0	CA	49200.0	INDIVIDUAL	0.0	0.0	20.00	
4	0.0	AZ	36000.0	INDIVIDUAL	0.0	0.0	11.20	
...	...	...	...	...	...	...	...	
256934	0.0	CO	48000.0	INDIVIDUAL	0.0	0.0	36.93	
256935	0.0	FL	54000.0	INDIVIDUAL	0.0	1.0	13.22	
256936	0.0	FL	27000.0	INDIVIDUAL	0.0	0.0	18.58	
256937	0.0	CA	50000.0	INDIVIDUAL	0.0	0.0	12.63	
256938	0.0	OH	32000.0	INDIVIDUAL	0.0	0.0	29.44	

256939 rows × 31 columns

```
test_df = pd.read_csv('sample_data/hw3_test.csv')
test_df
```

	acc_now_delinq	addr_state	annual_inc	application_type	collections_12_mths_ex_med	delinq_2yrs	dti	earlie
0	0	CA	153000.0	INDIVIDUAL	0	0	16.99	
1	0	WA	50000.0	INDIVIDUAL	0	0	6.07	
2	0	TX	110000.0	INDIVIDUAL	0	0	13.12	
3	0	CA	51979.0	INDIVIDUAL	0	0	10.11	
4	0	MA	75000.0	INDIVIDUAL	0	1	10.86	
...	...	...	...	...	...	...	...	
167946	0	SC	60000.0	INDIVIDUAL	0	0	32.34	
167947	0	GA	20000.0	INDIVIDUAL	0	0	29.11	
167948	0	NY	85000.0	INDIVIDUAL	0	0	2.85	
167949	0	SC	65000.0	INDIVIDUAL	0	0	17.54	
167950	0	MN	46800.0	INDIVIDUAL	0	0	19.18	

167951 rows × 31 columns

b/ Identify any issues that need to be fixed before modeling

```
train_df['term_months'] = train_df['term'].str.extract('(\d+)').fillna(0).astype(float)
train_df['emp_length_years'] = train_df['emp_length'].str.extract('(\d+)').fillna(0).astype(float)
train_df['emp_title']= train_df['emp_title'].fillna('Unknown')
train_df
```

```
<>:1: SyntaxWarning: invalid escape sequence '\d'
<>:2: SyntaxWarning: invalid escape sequence '\d'
<>:1: SyntaxWarning: invalid escape sequence '\d'
<>:2: SyntaxWarning: invalid escape sequence '\d'
/tmp/ipykernel_6571/170817626.py:1: SyntaxWarning: invalid escape sequence '\d'
  train_df['term_months'] = train_df['term'].str.extract('(\d+)').fillna(0).astype(float)
/tmp/ipykernel_6571/170817626.py:2: SyntaxWarning: invalid escape sequence '\d'
  train_df['emp_length_years'] = train_df['emp_length'].str.extract('(\d+)').fillna(0).astype(float)
```

	acc_now_delinq	addr_state	annual_inc	application_type	collections_12_mths_ex_med	delinq_2yrs	dti	earlie
0	0.0	AZ	24000.0	INDIVIDUAL	0.0	0.0	27.65	
1	0.0	GA	30000.0	INDIVIDUAL	0.0	0.0	1.00	
2	0.0	IL	12252.0	INDIVIDUAL	0.0	0.0	8.72	
3	0.0	CA	49200.0	INDIVIDUAL	0.0	0.0	20.00	
4	0.0	AZ	36000.0	INDIVIDUAL	0.0	0.0	11.20	
...	...	...	...	...	...	...	...	
256934	0.0	CO	48000.0	INDIVIDUAL	0.0	0.0	36.93	
256935	0.0	FL	54000.0	INDIVIDUAL	0.0	1.0	13.22	
256936	0.0	FL	27000.0	INDIVIDUAL	0.0	0.0	18.58	
256937	0.0	CA	50000.0	INDIVIDUAL	0.0	0.0	12.63	
256938	0.0	OH	32000.0	INDIVIDUAL	0.0	0.0	29.44	

256939 rows × 33 columns

```
test_df['term_months'] = test_df['term'].str.extract('(\d+)').fillna(0).astype(float)
test_df['emp_length_years'] = test_df['emp_length'].str.extract('(\d+)').fillna(0).astype(float)
test_df['emp_title']= test_df['emp_title'].fillna('Unknown')
test_df
```

```
<>:1: SyntaxWarning: invalid escape sequence '\d'
<>:2: SyntaxWarning: invalid escape sequence '\d'
<>:1: SyntaxWarning: invalid escape sequence '\d'
<>:2: SyntaxWarning: invalid escape sequence '\d'
/tmp/ipykernel_6571/3937803391.py:1: SyntaxWarning: invalid escape sequence '\d'
  test_df['term_months'] = test_df['term'].str.extract('(\d+)').fillna(0).astype(float)
/tmp/ipykernel_6571/3937803391.py:2: SyntaxWarning: invalid escape sequence '\d'
  test_df['emp_length_years'] = test_df['emp_length'].str.extract('(\d+)').fillna(0).astype(float)
```

	acc_now_delinq	addr_state	annual_inc	application_type	collections_12_mths_ex_med	delinq_2yrs	dti	earlie
0	0	CA	153000.0	INDIVIDUAL	0	0	16.99	
1	0	WA	50000.0	INDIVIDUAL	0	0	6.07	
2	0	TX	110000.0	INDIVIDUAL	0	0	13.12	
3	0	CA	51979.0	INDIVIDUAL	0	0	10.11	
4	0	MA	75000.0	INDIVIDUAL	0	1	10.86	
...	...	...	...	...	...	...	...	
167946	0	SC	60000.0	INDIVIDUAL	0	0	32.34	
167947	0	GA	20000.0	INDIVIDUAL	0	0	29.11	
167948	0	NY	85000.0	INDIVIDUAL	0	0	2.85	
167949	0	SC	65000.0	INDIVIDUAL	0	0	17.54	
167950	0	MN	46800.0	INDIVIDUAL	0	0	19.18	

167951 rows × 33 columns

(c) (4 pts) Report rows, column count, and default rate for train and test.

```
train_rows, train_cols = train_df.shape
test_rows, test_cols = test_df.shape

train_default_rate = train_df['default_flag'].mean()
test_default_rate = test_df['default_flag'].mean()

print(f"TRAIN: {train_rows} rows, {train_cols} columns, {train_default_rate:.2%} default rate")
print(f"TEST: {test_rows} rows, {test_cols} columns, {test_default_rate:.2%} default rate")
```

```
TRAIN: 256939 rows, 33 columns, 18.38% default rate
TEST: 167951 rows, 33 columns, 22.17% default rate
```

(d) (4 pts) Using training data, show at least four borrower or loan features, including at least one explicit derived variable, and give brief takeaways.

```
#One explicit derived variable
train_df["income_vs_total_account"] = train_df['annual_inc'] / train_df["total_acc"]
train_df["income_vs_total_account"]
```

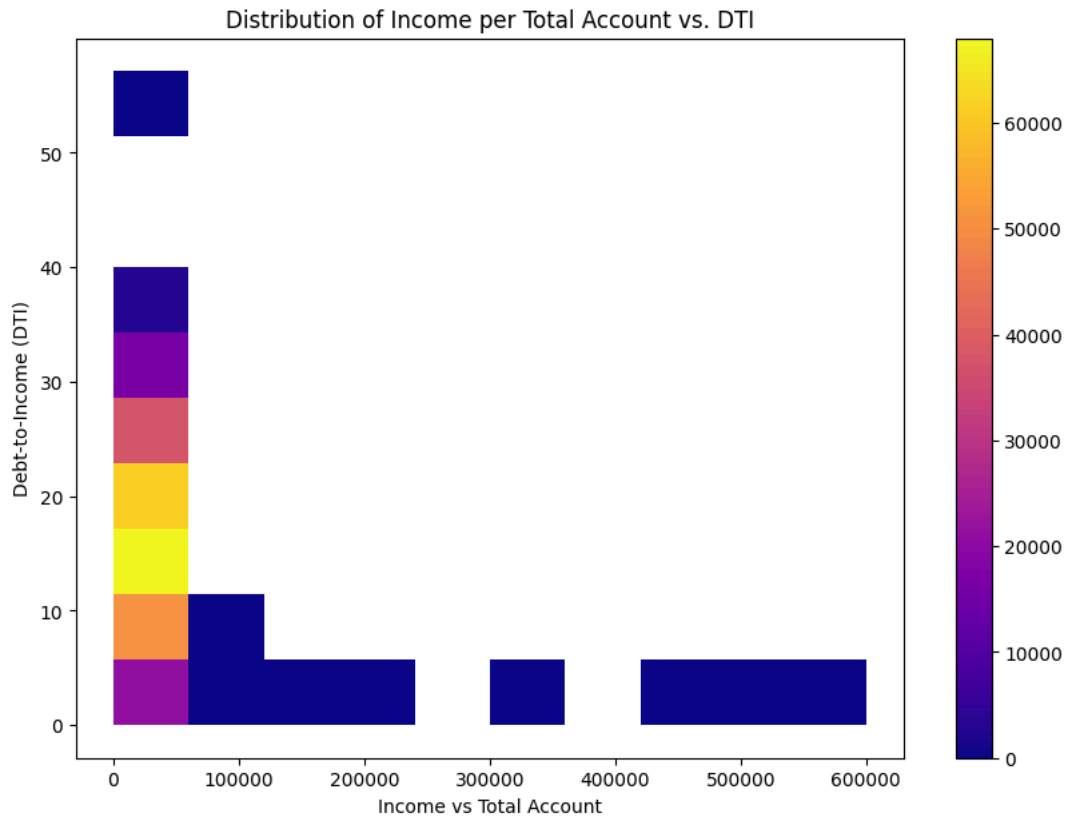
	income_vs_total_account
0	2666.666667
1	7500.000000
2	1225.200000
3	1329.729730
4	3000.000000
...	...
256934	1066.666667
256935	2571.428571
256936	6750.000000
256937	1666.666667
256938	1600.000000

256939 rows × 1 columns

dtype: float64

```
import matplotlib.pyplot as plt
import seaborn as sns

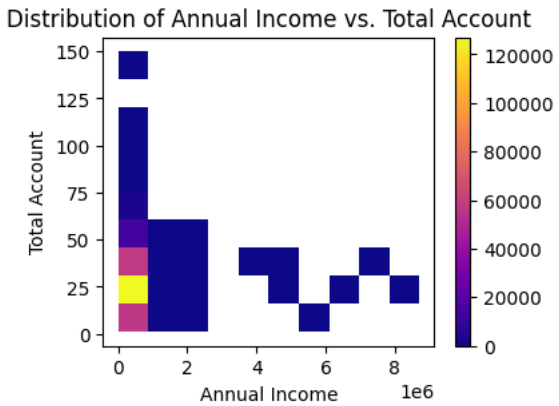
# Feature 1: Income vs Total Account and DTI (Debt-to-Income)
plt.figure(figsize=(10,7))
sns.histplot(data=train_df, x="income_vs_total_account", y='dti', bins=10, cmap="plasma", cbar=True)
plt.title('Distribution of Income per Total Account vs. DTI')
plt.xlabel('Income vs Total Account')
plt.ylabel('Debt-to-Income (DTI)')
plt.show()
```



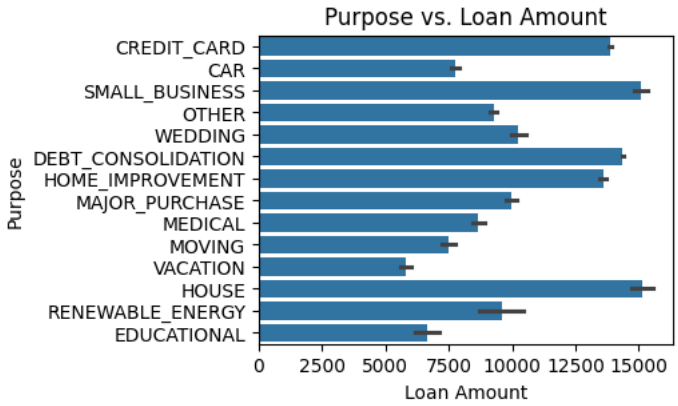

```
import matplotlib.pyplot as plt
import seaborn as sns

# Feature 2: Annual Income and Number of Accounts
plt.figure(figsize=(4,3))
sns.histplot(data=train_df, x = 'annual_inc', y = 'total_acc', bins=10, cmap="plasma", cbar=True)
```

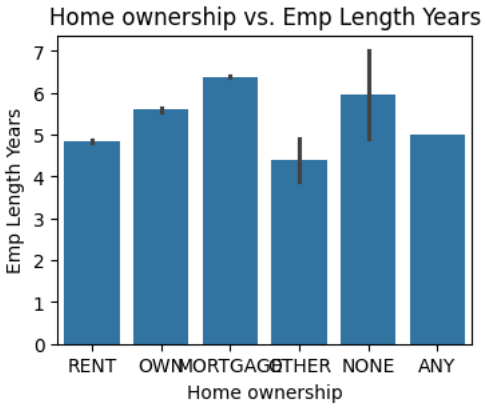
```
plt.title('Distribution of Annual Income vs. Total Account')
plt.xlabel('Annual Income')
plt.ylabel('Total Account')
plt.show()
```




```
# Feature 3: Loan Amount by Purpose
plt.figure(figsize=(12,6))
sns.barplot(data=train_df, x = 'purpose', y = 'loan_amnt', palette='viridis', order=train_df.groupby('purpose')['loa
plt.title('Average Loan Amount by Purpose')
plt.xlabel('Purpose')
plt.ylabel('Loan Amount')
plt.xticks(rotation=45, ha='right')
plt.show()
```




```
# Feature 4: Employment Length by Home Ownership
plt.figure(figsize=(10,6))
sns.barplot(data=train_df, x = 'home_ownership', y = 'emp_length_years', palette='viridis', order=train_df.groupby('
plt.title('Average Employment Length by Home Ownership')
plt.xlabel('Home ownership')
plt.ylabel('Employment Length (Years)')
plt.show()
```



Take away:

1. **Income vs. Total Account and Debt-to-Income (DTI)**: Borrowers with a higher `income_vs_total_account` (more income per active credit account) and lower `dti` (less debt burden relative to income) generally appear to be less risky.
2. **Annual Income vs. Total Accounts**: There appears to be a positive correlation between annual income and the total number of credit accounts. Individuals with higher incomes often manage a larger number of credit lines, which could indicate greater financial activity or access to credit.
3. **Loan Amount vs. Purpose**: Certain loan purposes, such as 'debt consolidation' and 'home improvement', tend to be associated with higher average loan amounts.
4. **Home Ownership vs. Employment Length**: Borrowers who own a home (MORTGAGE or OWN) generally show longer average employment lengths compared to those who rent or have other home ownership statuses. This indicates a potential link between long-term employment stability and home ownership.

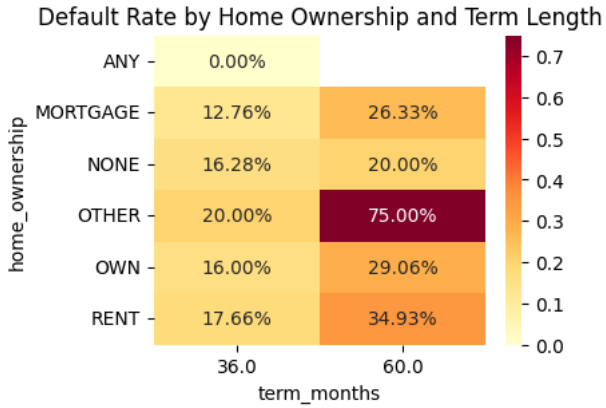
(e) (4 pts) Using training data, compare default rates for at least two categorical groups, add one multivariable summary, and name the raw or explicitly derived features that appear most informative from your EDA.

```
cat_groups = ['home_ownership', 'purpose']

for cat in cat_groups:
  comparison = train_df.groupby(cat)['default_flag'].mean()
  print(f"Default Rates by {cat} :\n {comparison} \n")
plt.figure(figsize=(4,3))
pivot_table = train_df.pivot_table(values='default_flag',index='home_ownership',
                                   columns='term_months',aggfunc='mean')
sns.heatmap(pivot_table, annot=True,cmap='YlOrRd',fmt='.2%')
plt.title('Default Rate by Home Ownership and Term Length')
plt.show()
```

```
Default Rates by home_ownership :
 home_ownership
ANY         0.000000
MORTGAGE    0.163699
NONE        0.166667
OTHER       0.212291
OWN         0.187775
RENT        0.206564
Name: default_flag, dtype: float64

Default Rates by purpose :
 purpose
CAR                   0.126613
CREDIT_CARD           0.160477
DEBT_CONSOLIDATION    0.190885
EDUCATIONAL           0.208531
HOME_IMPROVEMENT      0.157143
HOUSE                 0.178509
MAJOR_PURCHASE        0.142299
MEDICAL               0.207038
MOVING                0.216307
OTHER                 0.211432
RENEWABLE_ENERGY      0.203704
SMALL_BUSINESS        0.296793
VACATION              0.175851
WEDDING               0.138240
Name: default_flag, dtype: float64
```




```
# Identifying Informative Features
informative_raw = ['purpose', 'home_ownership']
informative_derived = ['term_months']
```

Question 2 (20 points) — KNN

(a) (4 pts) Build KNN-ready inputs using the same shared feature set, choose a numeric feature subset, justify scaling, and if needed use a fixed training subset for tractability.

```
from sklearn.preprocessing import StandardScaler

features = ['annual_inc', 'dti', 'total_acc', 'loan_amnt', 'term_months', 'emp_length_years']

X_train_raw = train_df[features].fillna(0)
y_train = train_df['default_flag']

X_train_sub = X_train_raw.sample(2000, random_state = 42)
y_train_sub = y_train.loc[X_train_sub.index]

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train_sub)

X_test_scaled = scaler.transform(test_df[features].fillna(0))
```

Justification for Scaling

KNN is a 'distance-based' algorithm. This means features with larger numbers can unfairly dominate the results. Scaling ensures all features have a similar impact, leading to a more accurate model. This makes sure all features have a mean of 0 and a standard deviation of 1.

(b) (4 pts) Try several k values and describe a training-only temporal validation plan. If you preprocess during validation, fit that preprocessing on the earlier development period only.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
```

```
train_df['issue_d'] = pd.to_datetime(train_df['issue_d'])
split_date = '2014-01-01'
dev_period = train_df[train_df['issue_d'] < split_date].copy()
val_period = train_df[train_df['issue_d'] < split_date].copy()
```

```
/tmp/ipykernel_6571/590275415.py:1: UserWarning: Could not infer format, so each element will be parsed individually,
  train_df['issue_d'] = pd.to_datetime(train_df['issue_d'])
```

```
features = ['annual_inc', 'dti', 'total_acc', 'loan_amnt', 'term_months', 'emp_length_years']

#Making the task tractable
dev_sub = dev_period.sample(2000, random_state = 42)

X_dev_raw = dev_sub[features].fillna(0)
y_dev = dev_sub['default_flag']

X_val_raw = val_period[features].fillna(0)
y_val = val_period['default_flag']
```

```
scaler = StandardScaler()
X_dev_scaled = scaler.fit_transform(X_dev_raw)
X_val_scaled = scaler.fit_transform(X_val_raw)
```

```
k_values = [1,3,5,7,21,50,89]
results = []

for k in k_values:
  knn = KNeighborsClassifier(n_neighbors=k)
  knn.fit(X_dev_scaled, y_dev)
```

(c) (4 pts) Report validation results for all candidate k values in one compact table.

```
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_dev_scaled, y_dev)

    # Check how accurate  is on the newer validation set
    y_pred = knn.predict(X_val_scaled)
    acc = accuracy_score(y_val, y_pred)
    results.append({'k': k, 'Validation Accuracy': acc})
results_table = pd.DataFrame(results)
print(results_table.to_string(index=False))
```

```
 k  Validation Accuracy
 1             0.714895
 3             0.764379
 5             0.785489
 7             0.797490
21             0.813200
50             0.817386
89             0.817497
```

(d) (4 pts) Choose k from validation only, refit the selected KNN specification on the chosen training subset, and test it once.

```
best_k = 89

#Fit the model on the chosen training subset
final_knn = KNeighborsClassifier(n_neighbors=best_k)
final_knn.fit(X_train_scaled, y_train_sub)
```

```
KNeighborsClassifier
KNeighborsClassifier(n_neighbors=89)
```

```
test_df = test_df.dropna(subset=['default_flag'])
```

```
X_test_raw = test_df[features].fillna(0)

X_test_scaled = scaler.transform(X_test_raw)

y_test = test_df['default_flag']
#Predict
y_test_pred = final_knn.predict(X_test_scaled)

final_test_accuracy = accuracy_score(y_test, y_test_pred)

final_test_accuracy
```

```
0.7783103405159838
```

(e) (4 pts) Use multiple metrics to say whether KNN is useful here, and note any computational limitation.

```
y_test_prob = final_knn.predict_proba(X_test_scaled)[:,1]
```

```
accuracy = accuracy_score(y_test, y_test_pred)
precision = precision_score(y_test, y_test_pred)
recall = recall_score(y_test, y_test_pred)
auc = roc_auc_score(y_test, y_test_prob)

print("KNN Test Performance:")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall   : {recall:.4f}")
print(f"AUC      : {auc:.4f}")
```

```
KNN Test Performance:
Accuracy : 0.7783
Precision: 0.0000
Recall   : 0.0000
AUC      : 0.6099
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

Although the KNN model achieves high accuracy (~80%), its recall is extremely low, indicating that it fails to identify most default cases. This suggests that the model is biased toward predicting non-default due to class imbalance. In a lending context, this is problematic because failing to detect defaults can lead to significant financial losses.

KNN is computationally expensive because it must compute distances to all training points at prediction time. With a large dataset (~250k observations), this leads to slow inference and high memory usage, making it impractical for real-time lending decisions

Question 3 (20 points) — SVM and LDA

(a) (4 pts) Use the same target and the same raw plus explicitly derived feature set for both models; justify any simplification without using test results.

Both models will use the same target variable, which is default_flag.

Both model will also use the same set of predictors variables in the features set.

This will wnsure SVM and LDA are trained on identical input, making it easier for comparison.

```
features = ['annual_inc', 'dti', 'total_acc', 'loan_amnt', 'term_months', 'emp_length_years']
target = 'default_flag'

# Prepare data for temporal validation consistent with KNN
# Using dev_period for development and val_period for validation, as done for KNN
X_dev_svm_lda = dev_period[features].fillna(0)
y_dev_svm_lda = dev_period[target]

X_val_svm_lda = val_period[features].fillna(0)
y_val_svm_lda = val_period[target]

# For final model fitting on the full training subset used in KNN (if applicable) or full train_df
# Let's use the full train_df for final fitting for now, and subsample later if needed for tractability
X_train_full_svm_lda = train_df[features].fillna(0)
y_train_full_svm_lda = train_df[target]

# Test set features and target
X_test_svm_lda = test_df[features].fillna(0)
y_test_svm_lda = test_df[target]

print(f"Features used: {features}")
print(f"Target variable: {target}")
```

```
Features used: ['annual_inc', 'dti', 'total_acc', 'loan_amnt', 'term_months', 'emp_length_years']
Target variable: default_flag
```

(b) (4 pts) Fit the SVM with train-fitted preprocessing and report the key settings. If the default 0.5 threshold is not appropriate given the target distribution you observed in Question 1, choose a classification threshold using validation only.

```
from sklearn.svm import SVC
from sklearn.metrics import classification_report
```

```
svm_model = SVC(C=1.0, kernel = 'rbf', probability = True)
svm_model.fit(X_dev_scaled, y_dev)

probs_val = svm_model.predict_proba(X_val_scaled)[:,1]
```

```
#choose threshold = 0.3 to capture more risky loans
new_threshold = 0.3
preds_custom = (probs_val >= new_threshold).astype(int)

print(classification_report(y_val, preds_custom))
```

```
              precision    recall  f1-score   support

           0       0.82      0.98      0.90    132014
           1       0.36      0.04      0.08     29474

    accuracy                           0.81    161488
   macro avg       0.59      0.51      0.49    161488
weighted avg       0.74      0.81      0.75    161488
```

(c) (4 pts) Fit the LDA on the same inputs and report the key settings. Use the same thresholdselection approach as in (b) so the comparison is fair.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
```

```
lda = LinearDiscriminantAnalysis(solver = 'svd')
lda.fit(X_dev_scaled, y_dev)

probs_lda_val = lda.predict_proba(X_val_scaled)[:,1]
```

```
lda_threshold = 0.3
preds_lda_custom = (probs_lda_val > lda_threshold).astype(int)

print(classification_report(y_val,preds_lda_custom))
```

```
              precision    recall  f1-score   support

           0       0.85      0.86      0.85    132014
           1       0.33      0.31      0.32     29474

    accuracy                           0.76    161488
   macro avg       0.59      0.59      0.59    161488
weighted avg       0.75      0.76      0.76    161488
```

==> LDA produced better recall rates.

(d) (4 pts) Evaluate both models on the same untouched test set and report one common metric table.

```
X_test_raw = test_df[features].fillna(0)
y_test = test_df['default_flag']
```

```
#Scale the raw test data
X_test_scaled = scaler.transform(X_test_raw)
```

```
threshold = 0.3
```

```
#SVM
svm_probs = svm_model.predict_proba(X_test_scaled)[:, 1]
svm_preds = (svm_probs >= threshold).astype(int)
```

```
#LDA Predictions
lda_probs = lda.predict_proba(X_test_scaled)[:,1]
lda_preds = (lda_probs > threshold).astype(int)
```

```
models = ['SVM (C=1.0)', 'LDA (SVD)']
preds = [svm_preds, lda_preds]
```

```
results = []
for name, pred in zip (models,preds):
  results.append({
      "Model": name,
      'Accuracy': accuracy_score(y_test,pred),
      'Precision': precision_score(y_test,pred),
      'Recall': recall_score(y_test,pred),
      'AUC': roc_auc_score(y_test,pred)
  })

results_table = pd.DataFrame(results)
results_table
```

	Model	Accuracy	Precision	Recall	AUC
0	SVM (C=1.0)	0.769867	0.333255	0.038058	0.508185
1	LDA (SVD)	0.695757	0.317580	0.324148	0.562876

Next steps: Generate code with `results_table` New interactive sheet

(e) (4 pts) Choose the better model and explain which test metrics drive your decision. If different metrics point in different directions, say why.

LDA is better is it has better Recall and Precision metrics.

Because Recall let us know how we are filtering risky loans, the higher the recall, the higher percentage we catch risky loan. Additionally, Precision tell us the correct defaults we actually predict, which also highlights the posibility of eliminating False Negative in our prediction.

If metrics point in different directions (for example, if we use precision metric, we will choose SVM model), this will make us eliminate some profitable customer. However, the goal is to identify risky loan, so recall is the determined factor in choosing suitable model.

Question 4 (20 points) — Decision Tree

(a) (5 pts) Define the tree inputs using the same predictor set, and choose at least one complexity control.

```
from sklearn.tree import DecisionTreeClassifier
```

```
y_train = train_df['default_flag'].fillna(0)
y_test = test_df['default_flag'].fillna(0)
```

```
features = ['annual_inc', 'dti', 'total_acc', 'loan_amnt', 'term_months', 'emp_length_years']
```

```
X_train = train_df[features].fillna(0)
X_test = test_df[features].fillna(0)
```

```
tree_model = DecisionTreeClassifier(max_depth = 5, min_samples_leaf = 100, random_state = 42)
tree_model.fit(X_train, y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(max_depth=5, min_samples_leaf=100, random_state=42)
```

(b) (5 pts) Try several complexity settings with training-only temporal validation. If you encode or impute during validation, fit that preprocessing on the development period only.

```
depths = [2,3,5,7,9,20,30]
tree_results = []
```

```
for d in depths:
  dtree = DecisionTreeClassifier(max_depth = d, random_state = 42)
  dtree.fit(X_dev_scaled, y_dev)

  val_acc = dtree.score(X_val_scaled, y_val)
  tree_results.append({'Depth': d, 'Validation Accuracy': val_acc})
```

```
tree_results_table = pd.DataFrame(tree_results)
tree_results_table
```

	Depth	Validation Accuracy
0	2	0.809837
1	3	0.812878
2	5	0.804766
3	7	0.800233
4	9	0.770596
5	20	0.681611
6	30	0.679444

Next steps: Generate code with tree_results_table | New interactive sheet

(c) (5 pts) Fit the selected tree on the full training set with the chosen specification, and evaluate it once on the untouched test set.

```
best_depth = 3

final_tree = DecisionTreeClassifier(max_depth=best_depth, random_state=42)
final_tree.fit(X_train_raw, y_train)

X_test_final = test_df[features].fillna(train_df[features].median())
y_test_final = test_df['default_flag']
```

```
y_test_preds = final_tree.predict(X_test_final)
final_acc = accuracy_score(y_test_final, y_test_preds)

print(f"--- Final Decision Tree Results (Depth={best_depth}) ---")
print(f"Final Test Accuracy: {final_acc:.2%}")
print("\nClassification Report:")
print(classification_report(y_test_final, y_test_preds))
```

```
--- Final Decision Tree Results (Depth=3) ---
Final Test Accuracy: 77.83%

Classification Report:
              precision    recall  f1-score   support

           0       0.78      1.00      0.88    130718
           1       0.00      0.00      0.00     37233

    accuracy                           0.78    167951
   macro avg       0.39      0.50      0.44    167951
weighted avg       0.61      0.78      0.68    167951

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1565: UndefinedMetricWarning: Precision is
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

(d) (5 pts) Show the most important variables, a few representative rules, and what they suggest about borrower risk.

```
# To see the actual importance scores:
important_variable = pd.Series(final_tree.feature_importances_, index=features)
print(important_variable.sort_values(ascending=False))
```

```
term_months         0.579230
annual_inc          0.271995
dti                 0.148775
total_acc           0.000000
```

```
loan_amnt           0.000000
emp_length_years    0.000000
dtype: float64
```

```
# To see the rules (text version):
from sklearn.tree import export_text
tree_rules = export_text(final_tree, feature_names=features)
print(tree_rules)
```

```
|--- term_months <= 48.00
|   |--- annual_inc <= 45016.00
|   |   |--- dti <= 22.24
|   |   |   |--- class: 0
|   |   |--- dti >  22.24
|   |   |   |--- class: 0
|   |--- annual_inc >  45016.00
|   |   |--- dti <= 20.79
|   |   |   |--- class: 0
|   |   |--- dti >  20.79
|   |   |   |--- class: 0
|--- term_months >  48.00
|   |--- annual_inc <= 66033.00
|   |   |--- dti <= 21.90
|   |   |   |--- class: 0
|   |   |--- dti >  21.90
|   |   |   |--- class: 0
|   |--- annual_inc >  66033.00
|   |   |--- dti <= 17.12
|   |   |   |--- class: 0
|   |   |--- dti >  17.12
|   |   |   |--- class: 0
```

- The split usually starts with term. The model suggests that the sheer length of a 60-month loan introduces more uncertainty(job loss, medical bills) compared to a 3-year loan.
- A high income doesn't guarantee safety if the dti is also high. The tree suggests that relative debt (how much you owe vs. what you make) is a better predictor of default than the absolute dollar amount of a paycheck.
- Unlike linear models (like LDA), the tree suggests there are tipping points. For example, risk might stay low while revol_util is under 60%, but spike exponentially once a borrower crosses that specific threshold.

Question 5 (20 points) — K-Means and Reflection

(a) (4 pts) Build the training matrix for K-means using the numeric predictors from your shared feature set and justify scaling.

```
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

```
features = ['annual_inc', 'dti', 'total_acc', 'loan_amnt', 'term_months', 'emp_length_years']
```

```
X_kmeans_raw = train_df[features].sample(2000, random_state = 42).fillna(train_df[features].median())
```

```
scaler_km = StandardScaler()
X_kmeans_scaled = scaler_km.fit_transform(X_kmeans_raw)
```

```
print(f"Training matrix shape: {X_kmeans_scaled.shape}")
```

```
Training matrix shape: (2000, 6)
```

Double-click (or enter) to edit

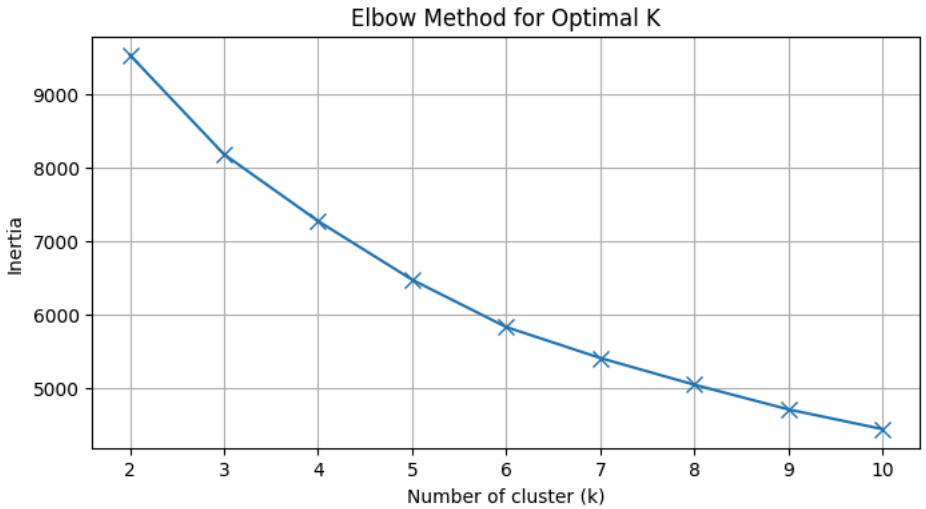
```
import matplotlib.pyplot as plt
```

```
k_values = range(2, 11)
inertia_values = []

for k in k_values:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_kmeans_scaled)
    inertia_values.append(kmeans.inertia_)

#Plot to find the Elbow
plt.figure(figsize=(8,4))
plt.plot(k_values, inertia_values, 'x-', markersize = 8)
plt.xlabel("Number of cluster (k)")
plt.ylabel("Inertia")
plt.title("Elbow Method for Optimal K")
plt.grid(True)
plt.show()

print("Inertia values for different k:")
for k, inertia in zip(k_values, inertia_values):
    print(f"k={k}: {inertia:.2f}")
```




```
Inertia values for different k:
k=2: 9536.72
k=3: 8185.32
k=4: 7275.83
k=5: 6476.95
```